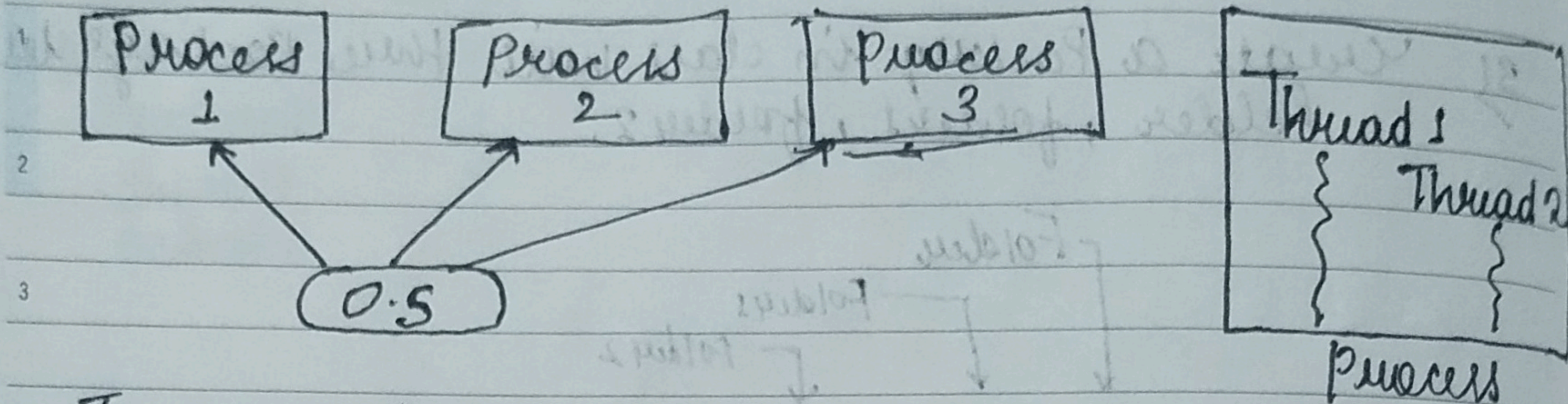


Chapter → 13Multithreading

Multiprocessing and Multithreading both are ^{used} ~~achieved~~ to achieve multitasking.



In a nut shell

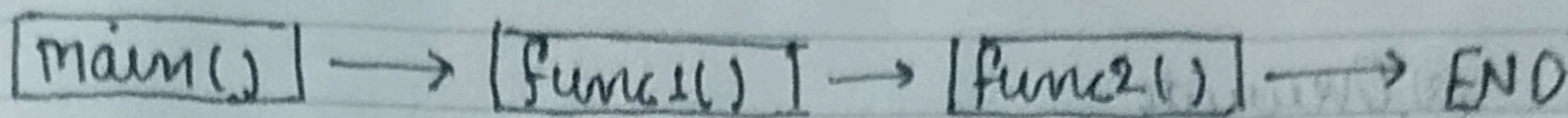
- Threads use shared memory area.
- Threads ⇒ Faster context Switching.
- A Thread is light-weight whereas a process is heavy weight.

For Examples : A word processor can have one Thread running in foreground as an Editor and another in the background auto saving the document.

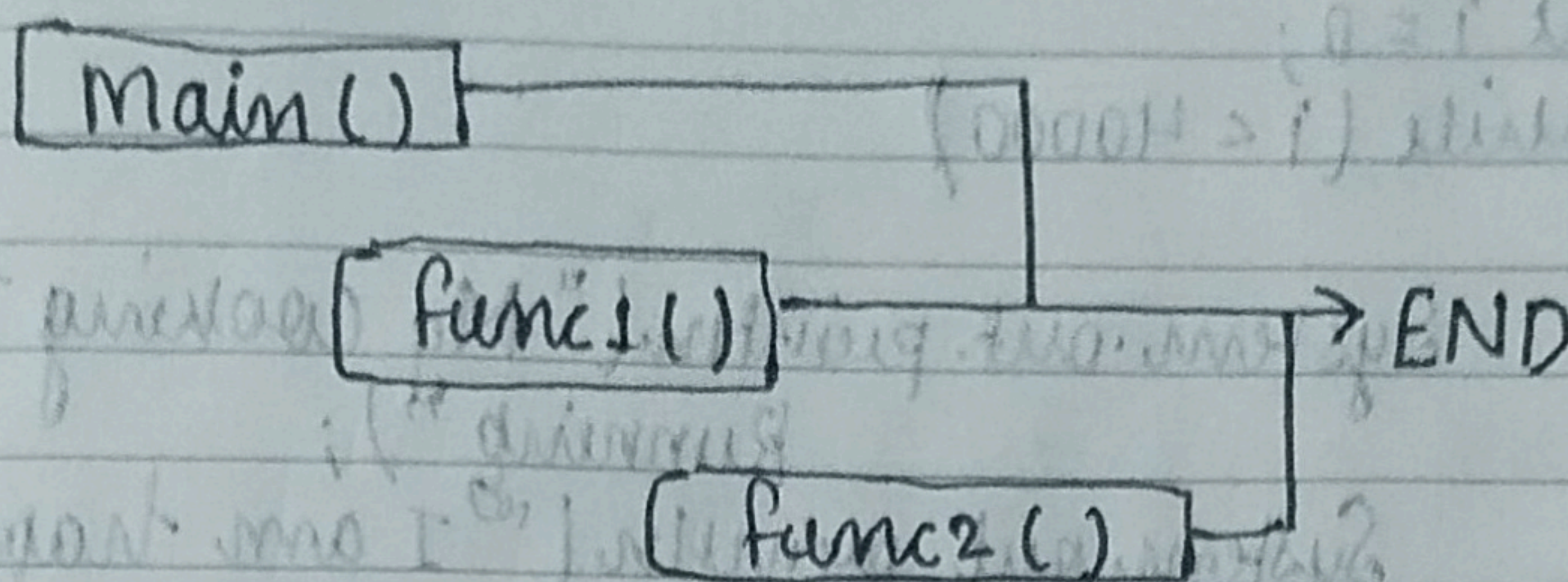
✍

Flow of control in Java

1) Without Threading:



2) With Threading:



Creating a Thread

There are two ways to create a thread in Java.

- 1) By extending Thread class
- 2) By implementing Runnable interface

~~Life cycle of a Thread~~

SUN 23

Examples : (By Extending Thread class)

class MyThread1 extends Thread

{

@Override

public void run()

{

int i = 0;

while (i < 40000)

{

System.out.println("My cooking Thread is Running");

System.out.println("I am happy!");

i++;

}

}

}

class MyThread2 extends Thread

{

@Override

public void run()

{

int i = 0;

while (i < 40000)

{

System.out.println("Thread 2 for chatting with here");

System.out.println("I am sad!");

i++;

}

}

}


```
public class THREADS
```

```
{
    public static void main(String [] args)
    {
        MyThread1 t1 = new MyThread1();
        MyThread2 t2 = new MyThread2();
        t1.start();
        t2.start();
    }
}
```

Examples : (By Runnable Interface)

```
class MyThreadRunnable1 implements Runnable
```

```
{
    public void run()
    {
        int i = 0;
        while (i < 4000)
        {
            System.out.println("I am a Thread 1 not a Thread 2");
            i++;
        }
    }
}
```

```
class MyThreadRunnable2 implements Runnable
{
```



```

9 public void run ()
10 {
11     int i = 0;
12     while (i < 4000)
13     {
14         System.out.println ("I am a Thread &
15                             not a thread &");
16     }
17 }
18 }
19 }

```

```

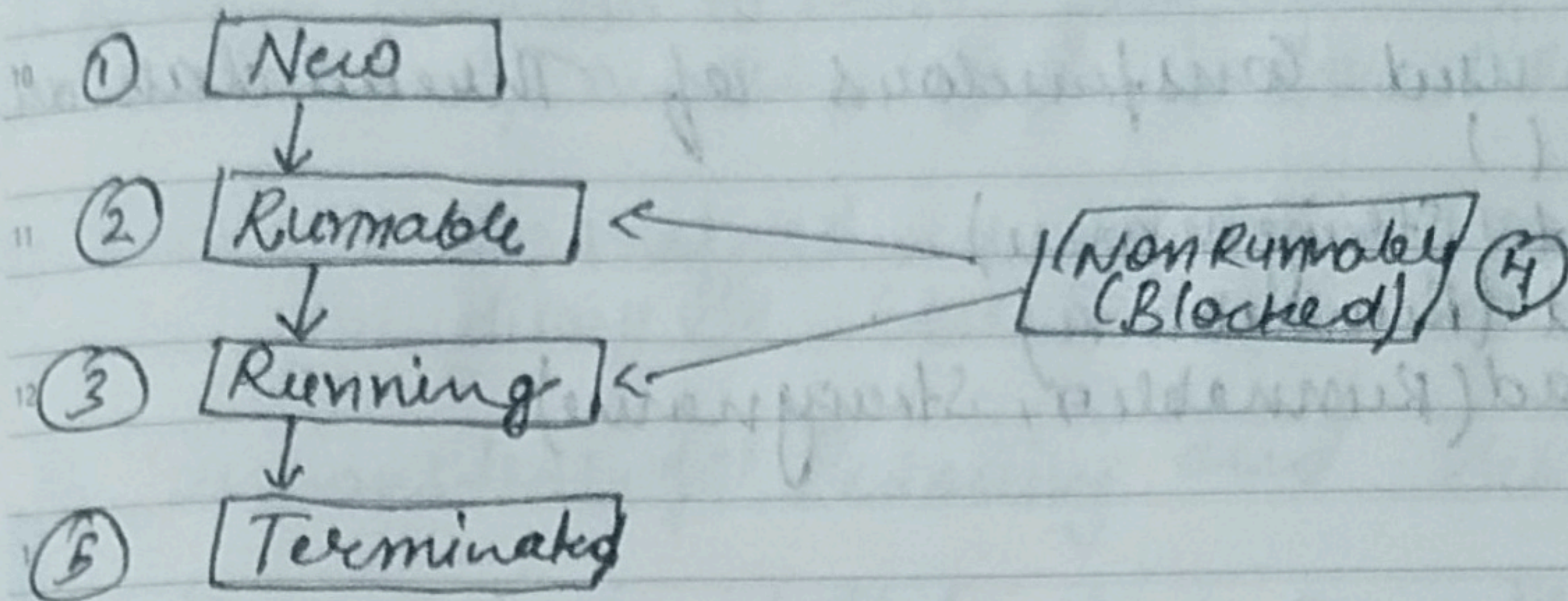
Public class RUNNABLE THREADS
{
    Public static void main (String[] args)
    {
        MyThreadRunnable1 bullet1 = new MyThread
        Runnable1();
        Thread gun1 = new Thread (bullet1);

        MyThreadRunnable2 bullet2 = new MyThread
        Runnable2();
        Thread gun2 = new Thread (bullet2);

        gun1.start ();
        gun2.start ();
    }
}

```


★ Life cycle of Thread



1) New → Instance of Thread created which is not yet started by invoking start().

2) Runnable → After invocation of start() & before it is selected to be run by the scheduler.

3) Running → After thread scheduler has selected it.

4) Non-Runnable → Thread alive, not eligible to run.

5) Terminated → run() method has ended.

The Thread Class

Commonly used constructors of Thread class are:

- 1) Thread()
- 2) Thread(String name)
- 3) Thread(Runnable r)
- 4) Thread(Runnable r, String name)

Examples:

```

class MyThr extends Thread
{
    public MyThr(String name)
    {
        super(name);
    }
    public void run()
    {
        while (true)
        {
            System.out.println("I am a Thread");
        }
    }
}

public class THREADCLASS
{
    public static void main (String [] args )
    {
        MyThr t = new MyThr("Harry");
        t.start();
        System.out.println("The ID of Thread" + t.getId());
    }
}
  
```